

MODULO 1 • GIORNO 1

Componenti Avanzati

Projection multi-slot e ng-template
Componenti dinamici e host directives
Integrazione librerie legacy (Splide)
Subject e BehaviorSubject



Corso Angular 21 Avanzato

Modulo 1 — Componenti Avanzati

Agenda — Modulo 1 (Pomeriggio G1)

Orario	Attività
11:30–13:00	Multi-slot, ng-template, componenti dinamici (schede 06–08)
13:00–14:00	Pranzo
14:00–14:30	Host directives (scheda 09)
14:30–15:30	Lab 1 — ProductCard multi-slot + quick-view
15:45–16:15	Libreria legacy + Subject/BehaviorSubject (scheda 10)
16:15–16:50	Lab 2 — NotificationService + carousel Splide

ng-content — Projection Multi-slot

```
<!-- product-card.html -->
<div class="card">
  <ng-content select="[card-badge]" />

  <h3>{{ product().name }}</h3>
  <ng-content />   <!-- slot default -->
  <footer>
    <ng-content select="[card-actions]">
      <button>Aggiungi</button> <!-- fallback -->
    </ng-content>
  </footer>
</div>

<!-- uso: --> <app-product-card>
  <span card-badge>Nuovo</span> </app-product-card>
```

Multi-slot

Ogni `<ng-content>` ha il suo select — attributo, tag o classe

Fallback

Contenuto di default se lo slot resta vuoto

Projection vs @Input()

@Input per i dati, projection per il markup

Nel Lab 1 la ProductCard viene rifattorizzata così

ng-template + ngTemplateOutlet

```
<!-- frammento inerte: non si renderizza da solo -->
<ng-template #rowTpl let-product>
  <div class="row">
    {{ product.name }} – {{ product.price }} €
  </div>
</ng-template>
<!-- montaggio -->
<ng-container
  [ngTemplateOutlet]="rowTpl"
  [ngTemplateOutletContext]="{ $implicit: product }" />
<!-- let-product riceve $implicit -->
```

```
// lato TS: ottenere il TemplateRef
@ViewChild('rowTpl')
rowTpl!: TemplateRef<unknown>;

// quando usarlo:

// - layout configurabile dal padre
// - righe/celle custom (liste, tabelle)
// - alternativa leggera ai componenti
//   dinamici quando serve solo markup
```

ng-content — proietta markup scritto dal padre, statico

ng-template — frammento riusabile, montato dove e quando vuoi

context — passa dati al template (let-var, \$implicit)

Componenti Dinamici — createComponent

```
// ancoraggio: ViewContainerRef
vcr = inject(ViewContainerRef);

open(product: Product) {

  const ref = this.vcr
    .createComponent(ProductQuickView);
  ref.setInput('product', product);
  ref.instance.closed
    .subscribe(() => ref.destroy());
}

// cleanup: la riga che si dimentica
ngOnDestroy() { this.vcr.clear(); }
```

```
// ProductQuickView (standalone)
@Component({
  selector: 'app-product-quick-view',
  template: `...`,
})
export class ProductQuickView {

  product = input.required<Product>();
  closed = output<void>();

}

// ComponentRef = handle:
// setInput, instance, destroy()
```

createComponent monta un componente a runtime — il Lab 1 lo usa per il quick-view del prodotto. Ricordare sempre destroy() / clear().

Host Directives

```
// direttiva riusabile
@Directive({ standalone: true })
export class Highlight {

  color = input('#FFEB3E');
}

// host directive sul componente
@Component({
  selector: 'app-product-card',
  hostDirectives: [{ directive: Highlight,
    inputs: ['color'] }],
})
export class ProductCard { }
```

Composizione

Il comportamento si monta sull'host, senza ereditarietà

Input/output esposti

Gli inputs/outputs della direttiva diventano dell'host

DI condivisa

La direttiva vive nell'injector del componente

Quando no: logica di business o markup → componente

Libreria Legacy — Wrapping + NgZone

```
// carousel.ts — wrapper di Splide
zone = inject(NgZone);
splide?: Splide;

ngAfterViewInit() {
  this.zone.runOutsideAngular(() => {
    this.splide = new Splide('.splide');
    this.splide.mount();
  });
}
ngOnDestroy() { this.splide?.destroy(); }
```

```
// rientrare in zona quando serve
this.splide.on('moved', (i) => {
  this.zone.run(() =>
    this.index.set(i));
});

// npm install @splidejs/splide
// CSS in angular.json o styles.css
// il Lab 2 wrappa Splide così
```

runOutsideAngular — gli eventi della libreria non intasano il change detection

zone.run — rientra in zona solo quando aggiorni lo stato Angular

ngOnDestroy — distruggere sempre l'istanza della libreria

Subject / BehaviorSubject — Comunicazione

```
// notification.service.ts
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private _toasts =

    new BehaviorSubject<Toast[]>([]);
  toasts$ = this._toasts.asObservable();

  show(message: string) {
    this._toasts.next(
      [...this._toasts.value,
        { message }]);
  }
}
```

```
// toast.ts – chi ascolta
svc = inject(NotificationService);
toasts = signal<Toast[]>([]);

constructor() {
  this.svc.toasts$

    .pipe(takeUntilDestroyed())
    .subscribe(t => this.toasts.set(t));
}

// Subject: bus eventi, nessun valore iniziale
// BehaviorSubject: conserva lo stato corrente
```

Subject/BehaviorSubject per comunicare tra componenti non imparentati; signal per lo stato locale. Il Lab 2 costruisce così i toast del carrello.

LAB 1

ProductCard multi-slot + Quick-view dinamico

14:30-15:30 (60 min)

- Rifattorizza ProductCard con ng-content multi-slot (badge, azioni)
- Crea ProductQuickView e montalo con createComponent
- setInput per il prodotto, destroy() alla chiusura
- Trigger dal bottone "anteprima" nella home

LAB 2

NotificationService + Carousel Splide

16:15–16:50 (35 min) — Semi-autonomo

```
// notification.service.ts
private _toasts =
  new BehaviorSubject<Toast[]>([]);
toasts$ = this._toasts.asObservable();

// carousel.ts

zone.runOutsideAngular(() => {
  new Splide('.splide').mount();
});

// npm install @splidejs/splide
```

Cosa costruisci

- NotificationService con BehaviorSubject — toast "aggiunto al carrello"
- Componente Toast con takeUntilDestroyed
- Carousel Splide wrappato con NgZone
- Integrazione nella home — delta su product-catalog-finale